Deliverable 1: Two Sets of C4 Context Diagrams

EventHub Ticketing. Two alternative C4 Context diagrams, each with its Structurizr DSL source file and rendered output file, followed by the pros and cons of each option.

Option A: Minimal Context

Source File (1-optionA-minimal-context.dsl)

Structurizr DSL. Paste this text into Structurizr Lite to regenerate the diagram.

```
workspace "EventHub Ticketing - Option A" "Minimal context: one actor, no external systems" {

    model {
        buyer = person "Ticket Buyer" "Student, parent, or fan anywhere in the world. The only user
role in version 1."

        eventhub = softwareSystem "EventHub Ticketing" "Python/Flask web app for registration,
browsing, and ticket purchase (simulated payment, on-screen confirmation). Event data, exchange-rate
config, and audit log live inside. BOUNDARY RULE: all incoming data is validated here." {
            tags "EventHub"
        }

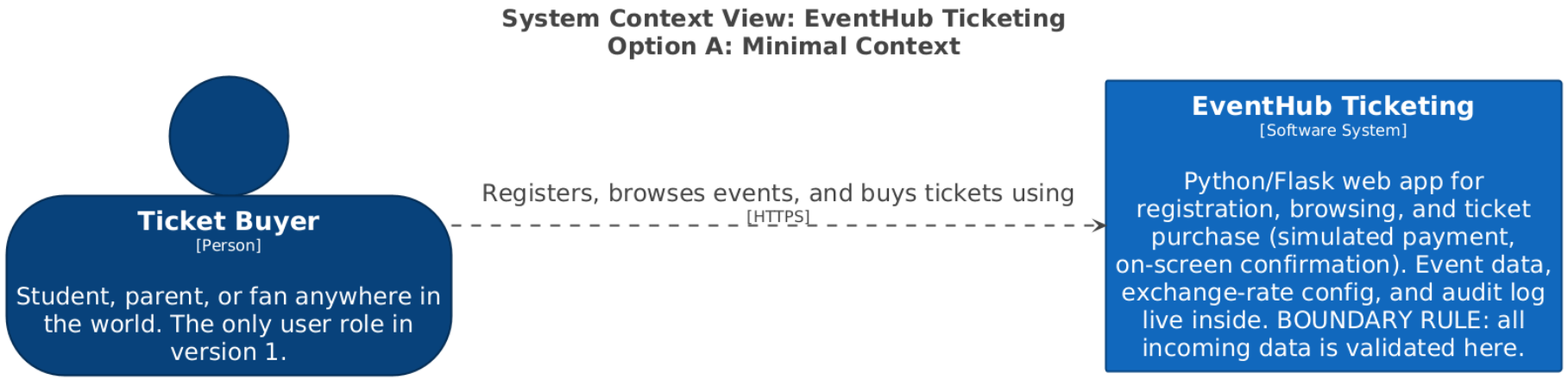
        buyer -> eventhub "Registers, browses events, and buys tickets using" "HTTPS"
    }

    views {
        systemContext eventhub "OptionA_Context" "Option A: Minimal Context" {
            include *
            autoLayout lr
        }

        styles {
            element "Person" {
                shape Person
                background #08427b
                color #ffffff
            }
            element "Software System" {
                background #1168bd
                color #ffffff
            }
        }
    }
}
```

Option A: Minimal Context: Output File

Rendered diagram generated from 1-optionA-minimal-context.dsl.



Option B: Operational Context

Source File (1-optionB-operational-context.dsl)

Structurizr DSL. Paste this text into Structurizr Lite to regenerate the diagram.

```
workspace "EventHub Ticketing - Option B" "Operational context: buyer plus operator, external
exchange-rate provider" {

    model {
        buyer = person "Ticket Buyer" "Student, parent, or fan anywhere in the world."

        operator = person "EventHub Operator" "Staff member who loads event data, updates exchange
rates, and reviews the audit log."

        eventhub = softwareSystem "EventHub Ticketing" "Python/Flask web app for registration,
browsing, and ticket purchase (simulated payment, on-screen confirmation). BOUNDARY RULE: all
incoming data is validated here." {
            tags "EventHub"
        }

        rateProvider = softwareSystem "Exchange Rate Provider" "External service that publishes
currency exchange rates." {
            tags "External"
        }

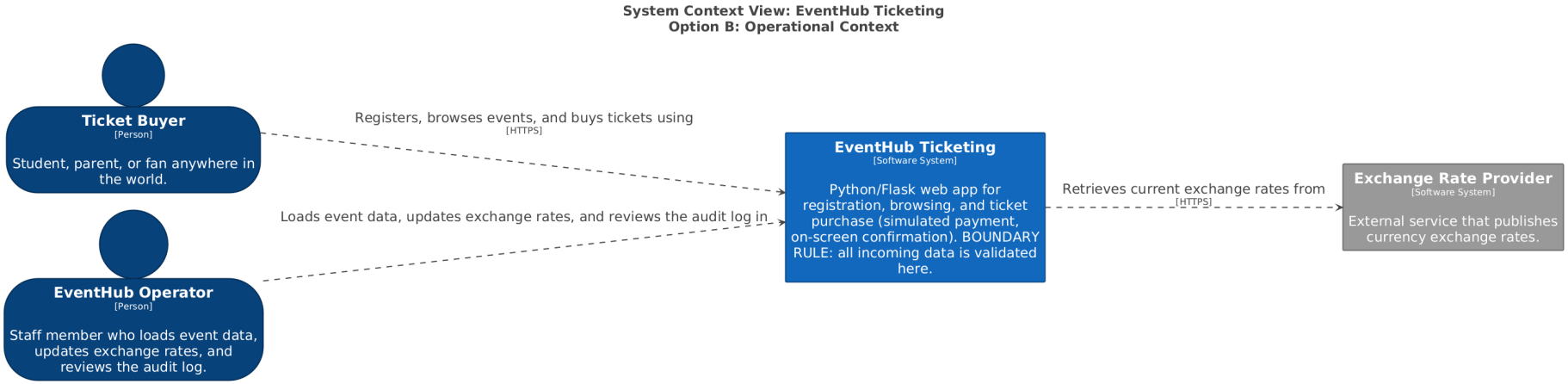
        buyer -> eventhub "Registers, browses events, and buys tickets using" "HTTPS"
        operator -> eventhub "Loads event data, updates exchange rates, and reviews the audit log in"
        eventhub -> rateProvider "Retrieves current exchange rates from" "HTTPS"
    }

    views {
        systemContext eventhub "OptionB_Context" "Option B: Operational Context" {
            include *
            autoLayout lr
        }

        styles {
            element "Person" {
                shape Person
                background #08427b
                color #ffffff
            }
            element "Software System" {
                background #1168bd
                color #ffffff
            }
            element "External" {
                background #999999
                color #ffffff
            }
        }
    }
}
```

Option B: Operational Context: Output File

Rendered diagram generated from 1-optionB-operational-context.dsl.



Pros and Cons of Each Option

Option A, the minimal context, has one actor, the Ticket Buyer. EventHub is the only system, and there are no external dependencies. The event data file, the exchange rate configuration, and the audit log all sit inside the system boundary.

Pros: It matches every decision made on Call 2 exactly. It is simple to read and simple to secure, because there is only one entry point. It also has no third-party availability or cost risk.

Cons: It does not show who maintains exchange rates or who reads the audit log. It leaves operational responsibilities invisible.

Option B, the operational context, adds an EventHub Operator person and an external Exchange Rate Provider.

Pros: It makes maintenance and audit responsibilities visible. It would also scale naturally if rates later become automated.

Cons: It contradicts Marcus's statement that there is no rate provider on the internet. It invents a role that nobody approved. It adds a second attack surface and an external dependency with no requirement behind it.